

数学实验与数学软件

第八课：MATLAB线性代数

谭军

中山大学数学学院

教师邮箱：mcstj@mail.sysu.edu.cn



矩阵分析与计算（符号数值通用）

- $\det(A)$ 计算A的行列式
- $\text{diag}(A)$ 取A对角元构成向量或利用向量A生成对角阵
- $\text{expm}(A)$ 计算A的指数矩阵，需将A特征值取指数复原
- $\text{inv}(A)$ 计算A的逆矩阵
- $\text{rank}(A)$ 计算A的秩
- $\text{tril}(A)$ 将A化为下三角矩阵（其余元素变为0）

- $[V \ D]=\text{eig}(A)$ 计算A的特征值保存到D，特征向量保存到V
- $[V \ J]=\text{jordan}(A)$ 计算A的jordan分解， $AV=VJ$
- $[U \ S \ V]=\text{svd}(A)$ 计算A的奇异值分解， $A=USV'$

- $[L \ U]=\text{lu}(A)$ 计算A的LU分解， $A=LU$
- $[Q \ R]=\text{qr}(A)$ 计算A的QR分解， $A=QR$

例2.10-1

【例2.10-1】求矩阵 $A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$ 的行列式、逆和特征根。

A=sym('A%d%d',[2,2]) %定义无具体值的符号型2x2矩阵的格式

A =
[A11, A12]
[A21, A22]

nA=ndims(A) %矩阵为2维矩阵（数组） nA = 2

SA=size(A) %矩阵的大小（规模） SA = 2 2

DA=det(A) DA = A11*A22 - A12*A21

IA=inv(A)
IA = [A22/(A11*A22 - A12*A21), -A12/(A11*A22 - A12*A21)]
 [-A21/(A11*A22 - A12*A21), A11/(A11*A22 - A12*A21)]

SIA=subexpr(IA,'ID') %公式式简化（本函数不考）

ID = 1/(A11*A22 - A12*A21)
SIA = [A22*ID, -A12*ID]
 [-A21*ID, A11*ID]

例2.10-1,2

【例2.10-1】求矩阵 $A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$ 的行列式、逆和特征根。

EA=subexpr(eig(A), 'D') %仅一个返回值时, eig将返回特征值

$D = (A_{11}^2 - 2*A_{11}*A_{22} + A_{22}^2 + 4*A_{12}*A_{21})^{(1/2)}$

$EA = \frac{A_{11}}{2} + \frac{A_{22}}{2} - \frac{D}{2}$
 $\frac{A_{11}}{2} + \frac{A_{22}}{2} + \frac{D}{2}$

【例2.10-2】（部分）Givens旋转（变换） $G = \begin{bmatrix} \cos t & -\sin t \\ \sin t & \cos t \end{bmatrix}$ 对矩阵 $A = \begin{bmatrix} \frac{\sqrt{3}}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} \end{bmatrix}$ 的旋转作用。

syms t

A=sym([sqrt(3)/2,1/2;1/2,sqrt(3)/2])

$A = \begin{bmatrix} 3^{(1/2)}/2, & 1/2 \\ 1/2, & 3^{(1/2)}/2 \end{bmatrix}$

G=[cos(t),-sin(t);sin(t),cos(t)]; %t的取值暂未给定

GA=G*A

GA =

$\begin{bmatrix} (3^{(1/2)} * \cos(t)) / 2 - \sin(t) / 2, & \cos(t) / 2 - (3^{(1/2)} * \sin(t)) / 2 \\ \cos(t) / 2 + (3^{(1/2)} * \sin(t)) / 2, & \sin(t) / 2 + (3^{(1/2)} * \cos(t)) / 2 \end{bmatrix}$

用solve函数解线性方程组符号解

【例】求 $d + \frac{n}{2} + \frac{p}{2} = q$, $n + d + q - p = 10$, $q + d - \frac{n}{4} = p$, $q + p - n - 8d = 1$ 线性方程组的符号解。

```
A=sym([1 1/2 1/2 -1;1 1 -1 1;1 -1/4 -1 1;-8 -1 1 1]);
```

```
b=sym([0;10;0;1]);%自行移项生成数字矩阵形式，再直接解掉
```

```
X1=A\b %比X1=inv(A)*b速度更快（MATLAB无需二次计算）
```

```
X1 =
```

```
1
```

```
8
```

```
8
```

```
9
```

```
syms d n p q
```

```
eq1= d+n/2+p/2-q ; eq2= d+n-p+q-10 ;
```

```
eq3= d-n/4-p+q ; eq4= -8*d-n+p+q-1 ;%跟课本相比，不会出警告
```

```
S=solve(eq1,eq2,eq3,eq4, d , n , p , q );%尽量不用字符串
```

```
disp([' d',' n',' p',' q'])
```

```
disp([S.d,S.n,S.p,S.q])
```

```
d n p q
```

```
[ 1, 8, 8, 9]
```

❑ **warning off** 可关闭所有的警告提示 (**warning on**恢复)

无穷多解时，新版本仅提供一个特解

【例】存在无穷多解的方程（组）会提供什么样形式的解

```
syms d n p q
eq1=d+n/2+p/2-q;
eq2=n+d+q-p-10;
eq3=q+d-n/4-p;
%3方程，4变量，不定方程
```

```
warning off %新版MATLAB这种方式反而不warning
S=solve(eq1,eq2,eq3,d,n,p,q);
```

```
disp(['    S.d','    S.n','    S.p','    S.q'])
disp([S.d,S.n,S.p,S.q]) %MATLAB定义z作为不定参数
    S.d    S.n    S.p    S.q
[ - 2,  8, - 4,  0] %注意：旧版本可以提供通解
```

数值线性代数-矩阵的范数

□ **norm(A)** 计算矩阵(或向量) A 的2-范数, 也称为谱范数

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2 = \sqrt{\lambda_{\max}(A^T A)}$$

其中向量 x 的2-范数即欧氏空间长度, $\lambda_{\max}$ 为 $A^T A$ 的最大特征值
因此谱范数也等于 A 的最大奇异值, 但不一定是最大特征值。

□ **norm(A, 'fro')** 计算矩阵 A 的Frobenius范数, 即把矩阵拉成一个长向量, 再计算其2-向量范数, 常用于矩阵误差分析

$$\|A\|_F = \sqrt{\sum_i (A^T A)_{i,i}} = \sqrt{\text{trace}(A^T A)}$$

□ **norm(A, 1)** 计算矩阵(或向量) A 的1-范数, 也称为列范数

$$\|A\|_1 = \max_{\|x\|_1=1} \|Ax\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|$$

其中向量 x 的1-范数为向量 x 所有元素绝对值的和。

矩阵的范数

□ `norm(A, inf)` 计算矩阵(或向量) A 的 ∞ -范数, 也称为行范数

$$\|A\|_{\infty} = \max_{\|x\|_{\infty}=1} \|Ax\|_{\infty} = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$$

其中向量 x 的 ∞ -范数即 x 所有元素的**最大绝对值**。

□ `sum(sum(A~=0.0))` 计算矩阵 A 的0-范数 (但这并不是真正的范数, 为什么), 含义为矩阵 A 的**非零元素的个数**。记为

$$\|A\|_0 = \#(A_{ij} \neq 0)$$

0-范数是对**矩阵稀疏性**的一种最直接、最准确的度量

□ `[U S V] = svd(A); norm(diag(S), 1)` 可计算矩阵 A 的核范数, 其定义为矩阵 A **所有奇异值的和**, 对核范数的约束是低秩逼近与低秩求解最常用的方法。

$$\|A\|_* = \sum_{i=1}^n \sqrt{\lambda_i(A^T A)}$$

低秩约束实例

- 复习：（强制降秩去噪模型）通过获取截取主成分来降秩：

$$\min_{A^*} \|A^* - B\|_F \text{ s.t. } \text{rank}(A^*) \leq 3$$

- 低秩惩罚项模型：通过对参数 $\lambda > 0$ 的大小调整，以及矩阵本身奇异值与0的逼近程度来自自动控制将秩的幅度：

$$\min_{A^*} \|A^* - B\|_F^2 + \lambda \cdot \text{rank}(A^*)$$

- 设 A^* 的奇异值为 $\vec{\sigma} = [\sigma_1, \sigma_2, \dots, \sigma_n]^\top$ ，该模型还可以表示为：

$$\min_{A^*} \|A^* - B\|_F^2 + \lambda \cdot \|\vec{\sigma}\|_0$$

- 松弛化低秩惩罚项模型：为了使模型具有更好的收敛性，可以将奇异值的 ℓ_0 范数更改为 ℓ_1 范数，对应以下模型：

$$\min_{A^*} \|A^* - B\|_F^2 + \lambda \cdot \|A^*\|_*$$

- 半正定核范数正则化问题，在 A^* 半正定时，其核范数等于迹：

$$\min_{A^*} \|A^* - B\|_F^2 + \lambda \cdot \text{trace}(A^*)$$

矩阵的其余标量特征-例4.2-1,2

□ 除行列式与秩外，矩阵的迹（方阵对角元的和）由 **trace(A)** 实现

【例4.2-1】矩阵标量特征函数

```
A=reshape(1:9,3,3)  A =
```

```
1         4         7
2         5         8
3         6         9
```

```
r=rank(A)
```

%显然秩为2（行列式为0，3行不成比例）

```
r = 2
```

```
d3=det(A)
```

```
d3 = 0
```

```
d2=det(A(1:2,1:2))
```

%A的前两行、前两列构成的子矩阵行列式

```
d2 = -3
```

```
t=trace(A)
```

%A的主对角元的和，若A不是方阵则报错

```
t = 15
```

【例4.2-2】矩阵标量特征函数的性质

```
format short, rng default  %短输出格式，随机种子至默认位置
```

```
A=rand(3,3);
```

%A和B为两个不同的3x3随机阵，元素服从[0,1]均匀分布

```
B=rand(3,3);
```

```
C=rand(3,4);
```

%C和D为两个随机非方阵，元素服从[0,1]均匀分布

```
D=rand(4,3);
```

例4.2-2

tAB=trace (A*B)

tAB = 3.7479

tBA=trace (B*A)

tBA = 3.7479

tCD=trace (C*D)

tCD = 3.3399

tDC=trace (D*C)

tDC = 3.3399

%同阶方阵或“内维”相等的矩阵相乘，交换次序迹不变

d_A_B=det (A) *det (B)

d_A_B = -0.0852

dAB=det (A*B)

dAB = -0.0852

dBA=det (B*A)

dBA = -0.0852

%经典代数结论 $|AB| = |BA| = |A| \cdot |B| = |B| \cdot |A|$

dCD=det (C*D)

dCD = -0.0557

dDC=det (D*C)

dDC = 1.5085e-017

%“内维”相等的矩阵相乘，交换次序行列式可能会改变

矩阵变换与特征值分解-例4.2-3

- `[R,ci] = rref(A)` 实现初等变换将A化为行阶梯型矩阵, 其中R储存了行阶梯型的矩阵, ci表示**阶梯元素**(线性独立)的列指标
- `null(A)` 得到A的零空间(核)的一组标准正交基
- `orth(A)` 得到A的值空间(像)的一组标准正交基

【例4.2-3】rref化矩阵为行阶梯型矩阵

`A=magic(4)`

A =

16	2	3	13
5	11	10	8
9	7	6	12
4	14	15	1

`[R,ci]=rref(A)` %A的秩等于3, 行阶梯型共有3层, 阶梯元素在前三列

R =

1	0	0	1
0	1	0	3
0	0	1	-3
0	0	0	0

ci =

1	2	3
---	---	---

例4.2-3

`r_A=length(ci)` %秩的另一种解读

```
r_A =  
      3
```

`aa=A(:,1:3)*R(1:3,4)`

%以R第四列为系数，利用A的前三列线性表示A的第四列

%注意到 $R(:,4)=R(:,1)+3*R(:,2)-3*R(:,3)=R(:,1:3)*R(1:3,4)$

%因此有 $A(:,4)=A(:,1)+3*A(:,2)-3*A(:,3)=A(:,1:3)*R(1:3,4)$

```
aa =  
     13  
      8  
     12  
      1
```

`err=norm(A(:,4)-aa)`

```
err =  
      0
```

%理论依据，初等行变换不改变线性方程组的解

%也不改变列向量组内部的相互线性表示、线性相关与线性无关的性质

%所以利用相同的线性表示的系数，我们可以用A的前三列线性表示A的第四列

例4.2-4

【例4.2-4】矩阵零空间及其含义

```
A=reshape(1:15,5,3);  
列)
```

%1~15按照5行3列重排（1~5在第一

```
X=null(A)
```

%零空间向量

```
X =
```

```
-0.4082
```

```
0.8165
```

```
-0.4082
```

```
S=A*X
```

%代入后应该得到零向量

```
S =
```

```
1.0e-014 *
```

```
0.2665
```

```
0.2665
```

```
0.3553
```

```
0.4441
```

```
0.6217
```

```
n=size(A,2);
```

%A的列数

```
l=size(X,2);
```

%x的列数（即零空间维数）

```
n-l==rank(A)
```

%判断矩阵秩是否等于（列数-零空间维数）

```
ans =
```

特征值分解-例4.2-5

【例4.2-5】 实矩阵的特征值分解

$A = \begin{bmatrix} 1 & -3 \\ 2 & 2/3 \end{bmatrix}$

A =

1.0000 -3.0000

2.0000 0.6667

$[V, D] = \text{eig}(A)$ %该矩阵有两个复特征根（D对角元），v为列特征向量构成的矩阵

V =

0.7746 0.7746

0.0430 - 0.6310i 0.0430 + 0.6310i

D =

0.8333 + 2.4438i 0

0 0.8333 - 2.4438i

$[VR, DR] = \text{cdf2rdf}(V, D)$

%将复数对角型转换为实数块对角型

(VR*DR/VR=A, 了解)

VR =

0.7746 0

0.0430 -0.6310

DR =

0.8333 2.4438

-2.4438 0.8333

特征值分解-例4.2-5

A1=V*D/V

%结果实际是相等的，但虚数部分有舍入误差

A1 =

```
1.0000 + 0.0000i   -3.0000 - 0.0000i
2.0000 - 0.0000i   0.6667  + 0.0000i
```

A1_1=real(A1)

A1_1 =

```
1.0000   -3.0000
2.0000    0.6667
```

A2=VR*DR/VR

A2 =

```
1.0000   -3.0000
2.0000    0.6667
```

err1=norm(A-A1, 'fro')

%误差分析

err1 =

```
4.6613e-016
```

err2=norm(A-A2, 'fro')

err2 =

```
4.4409e-016
```


MATLAB解线性方程组-例4.2-6

- $A \setminus b$ 是MATLAB最高效的解线性方程组的内建函数，此时 A 为 $m \times n$ 的矩阵， b 为 $m \times 1$ 的列向量。 $A \setminus B = \text{inv}(A) * B$ 可解矩阵方程
- MATLAB对于 A 列满秩且有解的情况将提供唯一解，对 A 列不满秩（或方阵 A 不可逆）且有解的情况将随机提供一个解。无解问题（或超定问题）将提供最小二乘解。

【例 4.2-6】求方程 $\begin{bmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{bmatrix} \cdot \mathbf{x} = \begin{bmatrix} 13 \\ 14 \\ 15 \\ 16 \end{bmatrix}$ 的解。

```
A=reshape(1:12,4,3);  
b=(13:16)';
```

%转置则化为列向量

```
ra=rank(A)
```

```
%ra = 2
```

```
rab=rank([A,b])
```

```
%rab = 2
```

%系数矩阵与增广矩阵的秩相等，且小于 A 列数，方程组存在无穷多解

例4.2-6

【例 4.2-6】求方程 $\begin{bmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{bmatrix} \cdot \mathbf{x} = \begin{bmatrix} 13 \\ 14 \\ 15 \\ 16 \end{bmatrix}$ 的解。

```
xs=A\b;
```

%求出方程组一个特解记为xs

警告：秩不足，秩 = 2，tol = 1.875718e-14。 %提示解不全，及误差大小

```
xg=null(A);
```

%求对应齐次方程零空间（基础解系）

```
c=rand(1);
```

%随机数

```
ba=A*(xs+c*xg)
```

%检验任意特解+对应齐次特解是否符合方程组

```
ba =
```

```
13.0000
```

```
14.0000
```

```
15.0000
```

```
16.0000
```

```
norm(ba-b)
```

%向量2-范数的误差分析,新版本误差变小

```
ans =
```

```
3.9721e-15
```

函数inv并不如左右除，例4.2-7

【例4.2-7】inv与左除的对比

```
rng default
```

```
A=gallery('randsvd',300,2e13,2); %300阶，cond=2e13，一个小奇异值  
%gallery命令的使用方法及其丰富，mathworks.cn上有十几页的使用方法大全
```

```
x=ones(300,1);
```

```
b=A*x;
```

```
cond(A)
```

```
%ans = 1.9997e+013
```

```
tic
```

```
xi=inv(A)*b;
```

```
%先求逆，再相乘浪费时间
```

```
ti=toc
```

```
%将当前计时存储在变量ti中
```

```
ti =
```

```
0.0273
```

```
eri=norm(x-xi)
```

```
%计算解的绝对误差
```

```
eri =
```

```
0.0957
```

```
rei=norm(A*xi-b)/norm(b)
```

```
%计算回代相对误差
```

```
rei =
```

```
0.0055
```

例4.2-7

【例4.2-7】 inv与左除的对比

```
tic;  
xd=A\b;
```

```
td=toc
```

%左除方法计算更快

```
td =  
    0.0110
```

```
erd=norm(x-xd)
```

```
erd =  
    0.0290
```

```
red=norm(A*xd-b)/norm(b)
```

```
red =  
    8.3734e-015
```

- 由此可见，在矩阵A条件数很大时，对应线性问题如果使用左除功能，不仅可以节省一些时间(新版本可能差不多)，而且可以大大降低回代后的相对误差。因此事实上，在大规模病态矩阵运算时，往往更加推荐左除算子，而不是inv函数。

线性方程组的具体解法算法举例

- ❑ **克拉默法则** $x_i = \frac{|D_i|}{|D|}$ 是一种常用来计算函数空间上线性方程组的解的方法，但对于向量空间的线性方程组，克拉默法则的理论效率则非常的低（算高阶行列式本身就是很大的负担），数值算法效率也比较低下。
- ❑ 在代数课程中，我们进行人工计算可以使用**初等行变换法**，将增广矩阵变化为行阶梯型矩阵（往往是上三角或类似形状），然后将方程组进行逐一回代，最终确定解向量所有元素 x_i 的值
- ❑ 为了使化简的结果更加美观，人工计算往往会进一步进行一系列的初等行变化，使增广矩阵变为**行最简形**，从而可以非常清晰的看出方程的唯一解或无穷多解的通解形式。

$$B = \begin{bmatrix} 2 & -1 & -1 & 1 & 2 \\ 1 & 1 & -2 & 1 & 4 \\ 4 & -6 & 2 & -2 & 4 \\ 3 & 6 & -9 & 7 & 9 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & -2 & 1 & 4 \\ 0 & 1 & -1 & 1 & 0 \\ 0 & 0 & 0 & 1 & -3 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & -1 & 0 & 4 \\ 0 & 1 & -1 & 0 & 3 \\ 0 & 0 & 0 & 1 & -3 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

线性方程组的数值迭代法

□ 从最初的线性方程组 $Ax = b$ 出发, 首先令 $A = M - N$, 然后有:
 $Mx = Nx + b$, 即 $x = M^{-1}Nx + M^{-1}b = Bx + f$.

其中, $B = M^{-1}N, f = M^{-1}b$

□ 数值迭代法会将上述等式右边的 x 代入上一步的解向量, 从而得出下一步的解向量, 如:

$$x^{(k+1)} = Bx^{(k)} + f$$

这种算法在 $\|B\| < 1$ 的情况下可保证收敛性

□ 根据 M, N 不同的选取, 可以衍生出不同的基本迭代算法

□ 实际的工程、建模问题中的矩阵 A 时常是大型稀疏矩阵, 因此迭代法的效率往往是远远高于传统方法的。特别是在图像处理的部分线性反问题 (矩阵 A 为千万X千万级) 中。

雅可比迭代法



采用矩阵分裂记号

$$\mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U}, \text{ 并有 } \mathbf{M} = \mathbf{D}, \mathbf{N} = \mathbf{L} + \mathbf{U}$$

其中

$$\mathbf{D} = \begin{bmatrix} \mathbf{a}_{11} & & & \\ & \mathbf{a}_{22} & & \\ & & \ddots & \\ & & & \mathbf{a}_{nn} \end{bmatrix}, \mathbf{L} = \begin{bmatrix} 0 & & & \\ -\mathbf{a}_{21} & 0 & & \\ \vdots & \ddots & \ddots & \\ -\mathbf{a}_{n1} & \cdots & -\mathbf{a}_{n,n-1} & 0 \end{bmatrix},$$
$$\mathbf{U} = \begin{bmatrix} 0 & -\mathbf{a}_{12} & \cdots & -\mathbf{a}_{1n} \\ & 0 & \ddots & \vdots \\ & & \ddots & -\mathbf{a}_{n-1,n} \\ & & & 0 \end{bmatrix}.$$

雅可比迭代法的矩阵表示形式为

$$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)} + \mathbf{D}^{-1}\mathbf{b} \stackrel{\Delta}{=} \mathbf{B}_J\mathbf{x}^{(k)} + \mathbf{f}$$

雅可比迭代法的数值实现

【例】雅可比迭代法解线性方程组，注意到该算法有很大的局限性

```
rng default
```

```
A=rand(400,400)+5e2*eye(400);
```

%雅可比迭代法需要对角元明显大于其他元素，否则一旦 $\|B\| \geq 1$ 就会使算法不收敛

```
x=rand(400,1);
```

```
b=A*x;
```

```
D=diag(diag(A)); %取出A的对角元
```

```
N = D-A; %N=L+U=D-A
```

```
B = D\N; f = D\b; % $x=Bx+f$ 雏形已确定
```

```
x_old = zeros(400,1); %迭代初值设为0向量
```

```
err = 1e12; %初始误差假设很大
```

```
iter = 0; %记录总迭代次数
```

```
while(err>1e-6) %当相邻两步绝对误差不超过 $1e-6$ 迭代即结束
```

```
    iter = iter + 1;
```

```
    x_new = B*x_old + f;
```

```
    err = norm(x_new-x_old); %记录该步结果与上一步结果的最新绝对误差
```

```
    x_old = x_new;
```

```
end
```

```
iter
```

```
iter = 19
```

%共迭代19次

```
err = norm(x_new-x)
```

```
2.5967e-07
```

%最终绝对误差

高斯-塞德尔迭代法

- 高斯-塞德尔迭代法的矩阵分裂方式有所不同，仍基于 $A = D - L - U$ ，但此时令 $M = D - L, N = U$ ，此时，迭代算法会相应的转化为

$$x^{(k+1)} = Bx^{(k)} + f = (D - L)^{-1}Ux^{(k)} + (D - L)^{-1}b$$

- 类似于雅可比迭代法，当矩阵的对角元拥有明显更大的绝对值时，高斯-塞德尔迭代法同样需要保证 $\|B\| < 1$ ，从而可以确保迭代算法的收敛性
- 在某些情况下，高斯-塞德尔迭代法的收敛要求更低。例如，当 A 的主对角元均为正值时，高斯-塞德尔迭代法只需 A 为正定矩阵即可，而雅可比迭代法则需要 A 与 $2D - A$ 均为正定矩阵
- 高斯-塞德尔迭代法的算法编辑与代码实现请同学们独立尝试完成。（不作为作业，考试要求读懂）

基于变分法解线性方程组

□ 二次函数的情形： $a > 0$ 时， $\operatorname{argmin}_{x \in \mathbb{R}} \frac{1}{2} ax^2 - bx$ 为 $ax = b$

□ 推广到矩阵：若 $A \succ 0$ (即矩阵 A 对称正定)，定义

$$\varphi(x) = \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle = \frac{1}{2} x^\top Ax - b^\top x$$

则线性方程组 $Ax = b$ 的解集等于 $\operatorname{argmin}_{x \in \mathbb{R}^n} \varphi(x)$ 的解集，将线性方程组求解问题转化为最优化问题的方法，称为变分法

□ 一元凸函数（二阶导大于0的函数）如 $\frac{1}{2} ax^2 - bx$ ，取到最小值的点 x^* 在抛物线底端，在任意一点，计算导数后，只需将 x 按照导数符号相反方向搜索即可直接获得唯一全局最优解 x^*

□ 由于矩阵 A 的正定性，最优化问题

$$\operatorname{argmin}_{x \in \mathbb{R}^n} \varphi(x) = \operatorname{argmin}_{x \in \mathbb{R}^n} \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle$$

为一个向量空间的凸问题。类似的，凸问题仅有一个全局最优解，并且可以通过梯度下降法（最速下降法）进行求解

变分-最速梯度下降法解线性方程组

- 光滑函数 $\varphi(x)$ 在 $x = x^{(k)}$ 下降最快的方向为其负梯度方向：
$$-\nabla\varphi(x^{(k)}) = -(Ax^{(k)} - b) = p^{(k)}$$

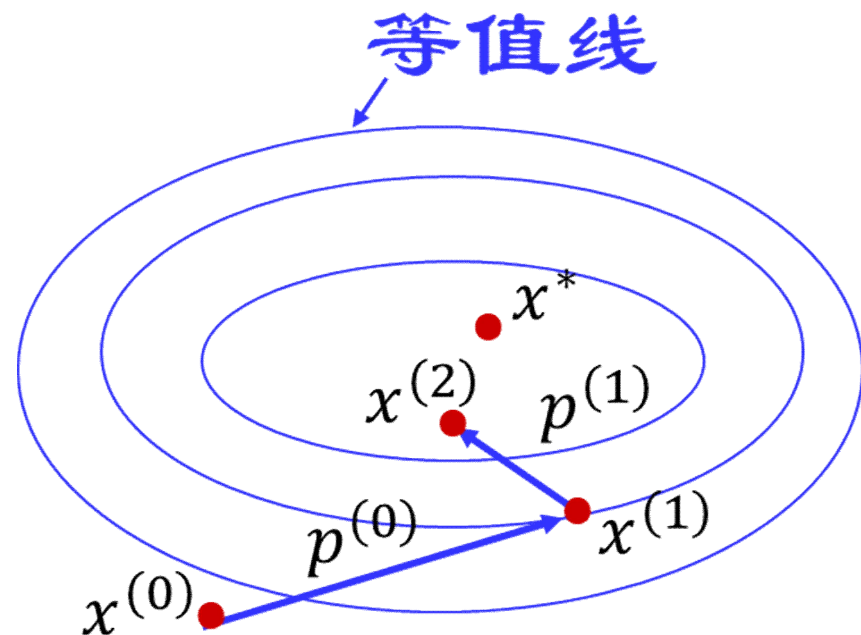
因此对于凸问题，只需将模型设计为 $x^{(k+1)} = x^{(k)} + \lambda p^{(k)}$ 即可使函数值不断降低，直到无限接近理论最小值。其中 $\lambda > 0$ 为待定步长，右下为梯度下降迭代示意图

- 步长 λ 不宜设为常数，因为迭代容易起初太慢，接近最优解时，又容易出现 $x^{(k)}$ 的抖动导致算法不收敛

- 最速下降法，即把每一步迭代在确定方向后，均转化为一个关于步长 λ 的一元函数问题
$$\lambda^{(k)} = \operatorname{argmin}_{\lambda} \varphi(x^{(k)} + \lambda p^{(k)})$$

- 最速步长

$$\lambda_k = \frac{\langle p^{(k)}, p^{(k)} \rangle}{\langle Ap^{(k)}, p^{(k)} \rangle}$$



变分-最速梯度下降法的实现

```
rng default,A0 = rand(300,300);  
A=A0+A0'+1e3*eye(300);           %A为一个随机的正定矩阵即可  
%在很多实际问题中，比雅可比，高斯-塞德尔方法更加可行  
x=rand(300,1);  
b=A*x;  
  
x_old = zeros(300,1);             %初始迭代值仍设为0向量  
err = 1e12;  
iter = 0;  
  
while(err>1e-6)                   %相邻两步误差不超过1e-6则终止迭代  
    iter = iter + 1;  
    p = b - A*x_old;               %迭代方向（目标函数负梯度）  
    lambda = (p'*p)/((A*p) '*p); %最速迭代步长  
    x_new = x_old + lambda*p;  
    err = norm(x_new-x_old); %统计迭代当前步的误差  
    x_old = x_new;  
end  
iter                               iter = 8                               %共迭代8次  
err = norm(x_new-x)               err = 9.2710e-08                   %最终绝对误差
```

变分-共轭梯度法解线性方程组

- 共轭梯度法是一种效率极高的解决正定线性方程组对应变分法问题的迭代算法。该方法与梯度下降法的主要区别就是每步的移动方向 $p^{(k)}$ 将会在 $P^{(k)} = \text{span}\{p^{(0)}, p^{(1)}, \dots, p^{(k-1)}, Ap^{(k-1)}\}$ 这一向量空间中进行最优化的选取，即

$$\varphi(x^{(k+1)}) = \min_{x \in P^{(k)}} \varphi(x)$$

- 对应迭代算法结构：

(1) 任取 $x^{(0)} \in \mathbb{R}$ ，计算 $r^{(0)} = b - Ax^{(0)}$ ， $p^{(0)} = r^{(0)}$ ， r 代表误差

(2) 对于 $k \in \mathbb{N}$ ，如下迭代：

$$\left\{ \begin{array}{l} \lambda_k = \frac{\langle r^{(k)}, r^{(k)} \rangle}{\langle Ap^{(k)}, p^{(k)} \rangle} \\ x^{(k+1)} = x^{(k)} + \lambda_k p^{(k)}, r^{(k+1)} = r^{(k)} - \lambda_k Ap^{(k)}, \\ \eta_k = \frac{\langle r^{(k+1)}, r^{(k+1)} \rangle}{\langle r^{(k)}, r^{(k)} \rangle}, p^{(k+1)} = r^{(k+1)} + \eta_k p^{(k)} \end{array} \right.$$

变分-共轭梯度法的实现 (了解)

```
rng default,A0 = rand(300,300);  
A=A0+A0'+1e3*eye(300);           %A为一个随机的正定矩阵即可  
x=rand(300,1);b=A*x;  
  
x_old = zeros(300,1); err = 1e12;iter = 0;  
r_old = b - A*x_old;              %初始反向方程组误差  
p = r_old;                        %初始迭代方向=反向梯度  
  
while(err>1e-6)                   %相邻两步x的误差不超过1e-6则终止迭代  
    iter = iter + 1;  
    lambda = (r_old'*r_old)/((A*p) '*p);           %x迭代步长  
    x_new = x_old + lambda*p;                       %对x迭代  
    err = norm(x_new-x_old);  
    r_new = r_old - lambda*A*p;                     %方程组误差的更新  
    eta = (r_new'*r_new)/(r_old'*r_old);            %迭代方向的变化  
    p = r_new + eta*p;                              %更新迭代方向  
    x_old = x_new;r_old = r_new;  
end  
iter = 5                                           %共迭代5次  
err = norm(x_new-x)                             err = 2.6935e-09 %绝对误差最小
```

使用MATLAB函数pcg完成共轭梯度法

- **`x = pcg(A,b)`** 可以实现带有简单病态预处理（实际上经常不太好使）的共轭梯度法求解 $Ax = b$ 的问题，矩阵 A 需要是一个对称正定的矩阵。此方法可以加入一些迭代终止条件 **`tol`** 等参数，也可以将 A 设置为函数句柄，用以代替模糊核操作等复杂线性变换，避免超大矩阵的乘法运算。

```
rng default,A0 = rand(300,300);  
A=A0+A0'+1e3*eye(300);      %A为一个随机的正定矩阵即可  
x=rand(300,1);b=A*x;
```

```
x_new = pcg(A,b);           %迭代默认终止误差  $1 \times 10^{-6}$   
pcg converged at iteration 4 to a solution with relative  
residual 3.3e-08.  
err = norm(x_new-x)         err = 4.0547e-07
```

```
x_new2 = pcg(A,b,1e-12);    %减小迭代中止误差  
pcg converged at iteration 7 to a solution with relative  
residual 1.1e-14.  
err = norm(x_new2 - x)      err = 1.4089e-13
```

第十一周电子版作业（下周二晚以前提交）

- ❑ 作业请于11月16日（周二）23:59:59或以前提交到课堂派平台。（请同学们加入正确的班级）
- ❑ 作业文件名格式：“05370038；李嘉；第十一周作业”
- ❑ 请将所有题目的方法设计（如果有）、代码、运行结果、绘图结果（如果有），放于同一个pdf（推荐使用）文件内（doc或docx也可）。文件名建议与邮件标题完全相同。
- ❑ 作业满分52分，电子版作业迟交者会按照每天5分的梯度损失分数。分数会根据答案的正确与否、算法的高效与否、文档展示的规范性等多个方面综合评定。
- ❑ 请勿抄袭，一经发现，最终总评不超过80。两次抄袭等于挂科。如果进行了部分方法与内容的借鉴，请在作业明示你的参考文献或借鉴人。

第十一周电子版作业

- Q1. 设矩阵 $A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 2 & 1 & -1 \end{bmatrix}$, 请用MATLAB函数计算矩阵A的1-范数, 2-范数, ∞ -范数, Frobenius范数, 核范数以及谱半径。
- Q2. 线性反问题 $A\vec{x} = \vec{b}$ 中, 病态矩阵A往往非常常见。请根据数值分析或其他相关资料。简单描述什么叫做矩阵的条件数, 以及高条件数病态矩阵对反问题求解的影响。
- 然后, 尝试用 `plot` 函数绘制1~50阶希尔伯特矩阵 (见第五课例6.1-3) 的条件数变化图, 证明其为病态矩阵。(提示: 病态矩阵使用常规方法计算条件数会不准)
- Q3. 习题4第10题, 使用MATLAB函数 `A\b`, `inv(A)*b` 与 `rref` 分别得到了什么结果, 为什么? 请解释原因。并且将线性方程组的通解表示出来。

求矩阵 $Ax = b$ 的解, A 为 4 阶魔方阵, b 是 (4×1) 的全 1 列向量。

课后反馈（不用交）

- 将今天讲过的例题尝试自己键入并运行一遍
- 思考题（**结论很重要**）：设 A 为 m 行 n 列实矩阵且 $m \leq n$ ，其奇异值为 $\vec{\sigma} = [\sigma_1, \sigma_2, \dots, \sigma_m]^T$ ，证明： $\|\vec{\sigma}\|_0 = \text{rank}(A)$ ， $\|\vec{\sigma}\|_2 = \|A\|_F$
- 阅读高等代数、数值分析与最优化的有关知识，理解矩阵的迹、行列式、**各种范数**、特征值分解等常用定义，温习线性方程组解的性质。并且理解**雅可比迭代法、高斯-塞德尔迭代法（需自行编码完成）、基于变分法的最速梯度下降法与共轭梯度法的算法过程**，能够利用MATLAB重现并完成简单的方程组求解（收敛性了解即可）。
- 阅读思考课本对应习题（不作为考点）

感谢同学们认真听课!

欢迎同学们积极提问、交流

mcstj@mail.sysu.edu.cn